

Task1

1.5) Open-Closed Principle (OCP) states that software entities should be open for extension, but closed for modification. If one were to use a createConnector(string input) function with an if/else chain, one would need to alter the code in order to add a new product. Which violates OCP because the code is not “closed for modification”. Modifying existing code risks introducing bugs or regressions into the logic that was working. In contrast, the Factory Method pattern allows one to extend the system by adding a new factory subclass without the need of altering the existing code.

1.6)

- > Connector - Abstract Product
- > CsvConnector - Concrete Product
- > ConnectorFactory - Abstract Creator
- > CsvFactory - Concrete Creator

Task 2

2.5)

- > A shallow copy, copies member variables directly as bitwise values whereas a deep copy allocates new memory on the heap for the pointer members and duplicates underlying data, making the clone independent of the original.
- > clone() should produce a deep copy.
- > The clone needs to be independent of the original so that changes could be made to the clone's internal state without altering the state of the registered prototype.

2.6)

- > The registry returns a clone() so each calling pipeline receives its own unique, isolated object to configure and execute independently.
- > If two pipelines share one prototype:
 - If two pipelines execute using the same prototype pointer, any internal state changes made by one pipeline will directly impact the other pipeline, leading to jumbled internal states.
 - The registry owns the registered prototypes and is responsible for deleting them in its destructor. If a pipeline was given the original pointer and deleted it upon completion, the registry's pointer would become a dangling pointer. Deletion by the registry or another pipeline would result in a double free error and undefined behaviour.

Task 3

3.5)

-> run() is a static function, static functions belong to the class rather than an object instance and do not have a this pointer. Since static functions lack an instance context, c++ does not allow making a function both static and virtual on the same method.

-> run() also acts as the Template Method in the Template Method design pattern. Its job is to define and enforce the fixed algorithm skeleton by executing functions in strict order. It is declared non-virtual to freeze the overarching execution sequence so subclasses cannot change the structure of the pipeline lifecycle.

-> A subclass that overrides run() could break the execution contract guaranteed by the base Pipeline class, potentially leading to uninitialised variable access, illegal state transitions, or total pipeline failure.

3.6)

-> run() - Template Method

-> connect() - Concrete Step

-> extract() - Primitive Operation

-> transform() - Concrete Step

-> load () - Primitive Operation

-> createCheckpoint() - Concrete Step/ Memento Hook

-> restore() - Concrete Step/ Memento Hook

Task 4

4.4)

-> The narrow interface is what the caretaker sees, it usually only has basic methods to pass the object around whereas the deep interface is what the organiser sees.

-> The originator uses the wide interface because it needs full access to the memento's internal variables to properly restore its state.

4.5)

-> During execution, after completing a processing stage, the pipeline calls createCheckpoint() to instantiate an AstaghfiruAllah RunCheckpoint memento containing its current states.

-> The generated snapshot is passed to the CheckpointManager, which calls save(checkpoint) to push the memento onto its history vector.

-> The pipeline proceeds to the load() stage and encounters a failure or crash before completion.

-> Upon recovery, the recovery mechanism asks the CheckpointManager for the last successful state by calling undo(), which pops and returns the most recent valid RunCheckpoint*.

-> The returned memento is passed back into the pipeline via restore(checkpoint), updating its stage and records back to their pre-failure state so the pipeline can safely resume without starting from scratch.

4.6)

-> Pipeline - Originator

-> RunCheckpoint - Memento

-> CheckpointManager - Caretaker